

CS3014 Estructuras de Datos Avanzadas

Laboratorio - Semana 10 - Sesión 1

Víctor Racsó Galván Oyola

vgalvan@utec.edu.pe

27 de mayo de 2026

¿Qué aprenderemos hoy?



¿Qué aprenderemos hoy?



- ▶ RMQ y LCA

¿Qué aprenderemos hoy?



- ▶ RMQ y LCA
- ▶ Pattern Matching

RMQ y LCA

RMQ



Range Minimum Query (Codeforces Group - Static RMQ)

RMQ



Range Minimum Query (Codeforces Group - Static RMQ)

Dada una secuencia A de n enteros, calcular el valor

$$\min_{l \leq i \leq r} \{A_i\}$$

para múltiples consultas.

RMQ



Range Minimum Query (Codeforces Group - Static RMQ)

Dada una secuencia A de n enteros, calcular el valor

$$\min_{l \leq i \leq r} \{A_i\}$$

para múltiples consultas.

Soluciones

RMQ



Range Minimum Query (Codeforces Group - Static RMQ)

Dada una secuencia A de n enteros, calcular el valor

$$\min_{l \leq i \leq r} \{A_i\}$$

para múltiples consultas.

Soluciones

Implementaremos primero una versión que tome $O(n \log n)$ de construcción y $O(1)$ de consulta.

RMQ



Range Minimum Query (Codeforces Group - Static RMQ)

Dada una secuencia A de n enteros, calcular el valor

$$\min_{l \leq i \leq r} \{A_i\}$$

para múltiples consultas.

Soluciones

Implementaremos primero una versión que tome $O(n \log n)$ de construcción y $O(1)$ de consulta.

Luego implementaremos una versión que tome $O(n)$ de construcción y $O(\log n)$ de consulta.

RMQ



Range Minimum Query (Codeforces Group - Static RMQ)

Dada una secuencia A de n enteros, calcular el valor

$$\min_{l \leq i \leq r} \{A_i\}$$

para múltiples consultas.

Soluciones

Implementaremos primero una versión que tome $O(n \log n)$ de construcción y $O(1)$ de consulta.

Luego implementaremos una versión que tome $O(n)$ de construcción y $O(\log n)$ de consulta.

Finalmente, una versión que tome $O(n)$ de construcción y $O(1)$ de consulta.

LCA



Lowest Common Ancestor

LCA



Lowest Common Ancestor

Dado un árbol con N nodos y raíz en el nodo 0, responder a Q consultas (u_i, v_i) con el LCA de u_i y v_i .

LCA



Lowest Common Ancestor

Dado un árbol con N nodos y raíz en el nodo 0, responder a Q consultas (u_i, v_i) con el LCA de u_i y v_i .

Soluciones

LCA



Lowest Common Ancestor

Dado un árbol con N nodos y raíz en el nodo 0, responder a Q consultas (u_i, v_i) con el LCA de u_i y v_i .

Soluciones

Implementaremos primero la solución con $O(N \log N)$ de construcción y $O(\log N)$ de consulta.

LCA



Lowest Common Ancestor

Dado un árbol con N nodos y raíz en el nodo 0, responder a Q consultas (u_i, v_i) con el LCA de u_i y v_i .

Soluciones

Implementaremos primero la solución con $O(N \log N)$ de construcción y $O(\log N)$ de consulta.

Podemos implementar fácilmente una solución con $O(N)$ de construcción y $O(\log N)$ de consulta.

LCA



Lowest Common Ancestor

Dado un árbol con N nodos y raíz en el nodo 0, responder a Q consultas (u_i, v_i) con el LCA de u_i y v_i .

Soluciones

Implementaremos primero la solución con $O(N \log N)$ de construcción y $O(\log N)$ de consulta.

Podemos implementar fácilmente una solución con $O(N)$ de construcción y $O(\log N)$ de consulta.

La implementación de la versión $O(N)$ de construcción y $O(1)$ de consulta queda como ejercicio.

Pattern Matching

Pattern Matching



Definición

Pattern Matching



Definición

Es un problema en el que se tiene una cadena T , llamada texto, y una cadena P , llamada patrón, y se desea saber si P ocurre en T como subcadena.

Pattern Matching



Definición

Es un problema en el que se tiene una cadena T , llamada texto, y una cadena P , llamada patrón, y se desea saber si P ocurre en T como subcadena. Su variación llamada *Multiple pattern matching* implica tener múltiples patrones $P_1, \dots, P_m$.

Pattern Matching



Definición

Es un problema en el que se tiene una cadena T , llamada texto, y una cadena P , llamada patrón, y se desea saber si P ocurre en T como subcadena. Su variación llamada *Multiple pattern matching* implica tener múltiples patrones $P_1, \dots, P_m$.

Aplicaciones

Pattern Matching



Definición

Es un problema en el que se tiene una cadena T , llamada texto, y una cadena P , llamada patrón, y se desea saber si P ocurre en T como subcadena. Su variación llamada *Multiple pattern matching* implica tener múltiples patrones $P_1, \dots, P_m$.

Aplicaciones

Búsquedas exactas de texto, motores de búsqueda, autocompletar.

Pattern Matching



Definición

Es un problema en el que se tiene una cadena T , llamada texto, y una cadena P , llamada patrón, y se desea saber si P ocurre en T como subcadena. Su variación llamada *Multiple pattern matching* implica tener múltiples patrones $P_1, \dots, P_m$.

Aplicaciones

Búsquedas exactas de texto, motores de búsqueda, autocompletar.
Búsqueda inexacta, búsqueda con wildcards, etc.

Escenarios



P conocido, T por conocer

Escenarios



P conocido, T por conocer

Consideraremos un escenario en el que se nos da P inicialmente, procesaremos P y luego se nos da T para hacer el emparejamiento.

Escenarios



P conocido, T por conocer

Consideraremos un escenario en el que se nos da P inicialmente, procesaremos P y luego se nos da T para hacer el emparejamiento.

Algoritmos: Knuth-Morris-Pratt, Z, Rabin-Karp, Aho-Corasick (multiple).

Escenarios



P conocido, T por conocer

Consideraremos un escenario en el que se nos da P inicialmente, procesaremos P y luego se nos da T para hacer el emparejamiento.

Algoritmos: Knuth-Morris-Pratt, Z, Rabin-Karp, Aho-Corasick (multiple).

T conocido, P por conocer

Escenarios



P conocido, T por conocer

Consideraremos un escenario en el que se nos da P inicialmente, procesaremos P y luego se nos da T para hacer el emparejamiento.

Algoritmos: Knuth-Morris-Pratt, Z, Rabin-Karp, Aho-Corasick (multiple).

T conocido, P por conocer

Consideraremos un escenario en el que se nos da T inicialmente, procesaremos T y luego se nos da P para hacer el emparejamiento.

Escenarios



P conocido, T por conocer

Consideraremos un escenario en el que se nos da P inicialmente, procesaremos P y luego se nos da T para hacer el emparejamiento.

Algoritmos: Knuth-Morris-Pratt, Z, Rabin-Karp, Aho-Corasick (multiple).

T conocido, P por conocer

Consideraremos un escenario en el que se nos da T inicialmente, procesaremos T y luego se nos da P para hacer el emparejamiento.

Algoritmos: Suffix array, Suffix tree, Suffix automaton.

Resumen de la sesión



Resumen de la sesión



- Implementamos RMQ

Resumen de la sesión



- ▶ Implementamos RMQ
- ▶ Implementamos LCA

Resumen de la sesión



- ▶ Implementamos RMQ
- ▶ Implementamos LCA
- ▶ Aprendimos de qué trata el problema de Pattern Matching

Gracias